



Lecture 14

Microservice Build

Sharif University of Technology
Winter 2026



Learning Objectives

What CI really means — and how teams get it wrong

Build pipelines: fast feedback from code to production

Continuous Delivery vs. Continuous Deployment

Artifact creation best practices

Monorepo vs. Multirepo — trade-offs & recommendations



Part I

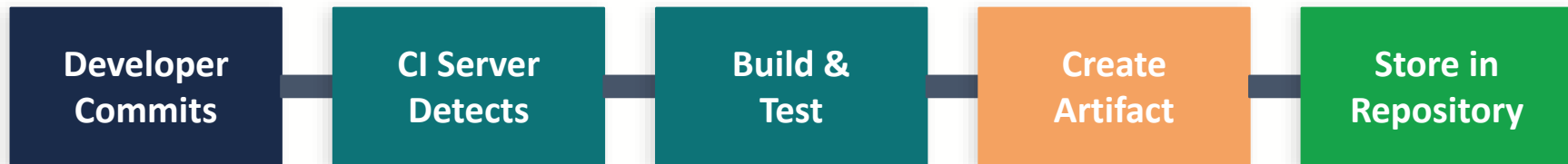
CI/CD

Fast feedback, reproducible releases, confident deployment

What Is Continuous Integration?



The core goal of CI is to keep everyone in sync by frequently integrating newly written code with the existing codebase.



- A CI server detects a commit, checks out the code, compiles it, and runs tests
- Integration should happen at least daily — ideally multiple times per day
- Artifacts produced are stored for reuse across all downstream stages
- CI gives fast feedback: broken tests caught within minutes, not days

3-Question CI Test



1 Do you check in to mainline once per day?

1

Code must integrate with everyone else's changes frequently. Even with short-lived branches, integrate into a single mainline at least daily.

2 Do you have a test suite to validate your changes?

2

Without tests, you only know syntactic integration worked — not whether system behaviour is broken. CI without verification is not CI.

3 When the build is broken, is fixing it the #1 priority?

3

A red build means the last change may not have integrated. Stop all further check-ins unrelated to the fix. Stacking changes drastically extends fix time.

Feature Branching vs. Trunk-Based Development



Feature Branching

- Separate branch per feature seems logical
- You are NOT regularly integrating your changes
- Fundamentally delays integration
- Results in a much more complex merge later
- Popular with open source (GitFlow model)
- Not ideal for typical day-to-day dev teams

Trunk-Based Development

- Everyone commits to one main branch
- Feature flags hide incomplete work
- Integration happens continuously
- Branches < 1 day lifetime if used at all
- Build breaks surface immediately
- Recommended by Newman & DORA research

Rule of thumb: Integrate early, integrate often. If using branches, keep them short-lived!

What Does the Research Say?



2016 State of DevOps Report (DORA + Puppet) — studied thousands of organizations:

"Branches or forks with very short lifetimes (less than a day) before being merged into trunk, and less than three active branches in total, are important aspects of continuous delivery and contribute to higher performance."

< 1 day

Branch lifetime

high-performing teams

< 3

Active branches

at any given time

Daily

Merges to trunk

or more frequent

- Open source projects (GitFlow model) differ from day-to-day closed-source development
- Small, readable patchsets are merged faster than large 'patch bombs' — true for both models

Feature Flags: Enabling Trunk-Based Development



- Commit half-finished work to trunk daily
- Flag off = feature invisible to users
- Gradually enable for % of users (progressive delivery)
- Quick rollback: just flip the flag



```
1 import os
2 from fastapi import FastAPI
3
4 app = FastAPI()
5 NEW_CHECKOUT_ENABLED = os.getenv("FEATURE_NEW_CHECKOUT", "false") == "true"
6
7 @app.get("/checkout")
8 def checkout():
9     if NEW_CHECKOUT_ENABLED:
10         return new_checkout_flow()
11     return legacy_checkout_flow()
12
13 # In production: FEATURE_NEW_CHECKOUT=false
14 # In staging:    FEATURE_NEW_CHECKOUT=true
```

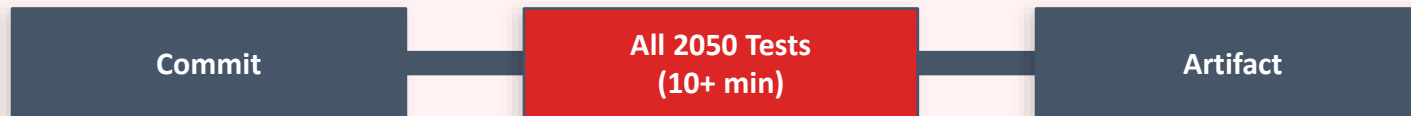
⚠ Trade-off: Feature flags add complexity. Clean them up once a feature is fully rolled out — stale flags become technical debt.

Build Pipelines: Why Not Run All Tests Together?

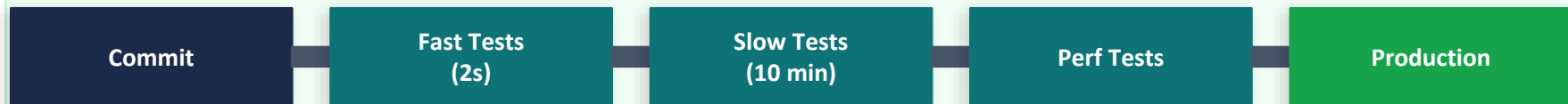


Problem: You have 2,000 fast unit tests (2s) and 50 slow integration tests (10 minutes). What happens if you run them all at once?

✗ Naive approach — all tests in one stage:



✓ Pipeline approach — fast feedback first:



If fast tests fail → stop immediately. No point waiting 10 minutes for integration tests.

Artifact Creation



Rule 1

Build Once, Only Once

Building multiple times wastes CI time, and different builds of the same commit can produce subtly different binaries depending on environment, compiler flags, or dependency resolution.



Rule 2

The Artifact You Test = The Artifact You Deploy

If you rebuild for production after testing in staging, you cannot guarantee the production binary is identical to what was validated. Don't.



Rule 3

Keep the Artifact Environment-Agnostic

Any config that varies between environments (DB URL, log level, feature flags) must live outside the artifact — in env vars, config maps, or a secrets manager.

Artifact types: Docker image (most common for microservices), JAR/WAR, npm package, OS package (deb/rpm), virtual machine image.

Artifact in Practice: Docker + FastAPI

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 8000
7 CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

```
1 # Build once, tag with commit SHA
2 docker build -t catalog:$(git rev-parse --short HEAD) .
3
4 # Push to registry
5 docker push registry.example.com/catalog:abc1234
6
7 # Staging deploy (same image, different env):
8 docker run -e DB_URL=postgres://staging/catalog \
9     registry.example.com/catalog:abc1234
10
11 # Production deploy (same image!):
12 docker run -e DB_URL=postgres://prod/catalog \
13     registry.example.com/catalog:abc1234
```

Environment-Agnostic Config in Python

```
1 import os
2 from pydantic_settings import BaseSettings
3
4 class Settings(BaseSettings):
5     database_url: str
6     log_level: str = "INFO"
7     feature_new_checkout: bool = False
8     max_connections: int = 10
9
10     class Config:
11         env_file = ".env"
12         env_file_encoding = "utf-8"
13
14 settings = Settings()
15
16 from fastapi import FastAPI
17 import logging
18 from config import settings
19
20 logging.basicConfig(level=settings.log_level)
21 app = FastAPI()
```

- Same Docker image
- Different .env or k8s ConfigMap per environment
 - Staging: LOG_LEVEL=DEBUG
 - Production: LOG_LEVEL=INFO
- No rebuild needed to change config
- Audit trail: config changes in Git, not in image

⚠ Never store secrets (passwords, API keys) in env files committed to Git! Use a secrets manager: AWS Secrets Manager or Kubernetes Secrets.

Continuous Delivery vs Continuous Deployment

Continuous Delivery

- Every check-in treated as a release candidate
- Pipeline assesses production readiness
- Humans can decide when/whether to deploy
- Continual focus on optimising path to production
- Compatible with manual approval gates

Continuous Deployment

- Extension of continuous delivery
- All passing check-ins deployed automatically
- Requires fully automated verification
- No human intervention in release
- Not right for everyone — but a valid goal

Key Rule: You cannot do Continuous Deployment without Continuous Delivery. But you *can* do Continuous Delivery without Continuous Deployment.

Why Does the Build Process Matter?



With a monolith, one bad build blocks one team. With microservices, a broken build pipeline can cascade across dozens of services.



Without CI

Developers integrate code infrequently → painful merge conflicts



Without CD pipelines

No visibility of which version is safe to ship



Without artifact discipline

"Works on my machine" — the classic excuse



With proper build practices

Fast feedback, reproducible releases, confident deployment



Part II

SOURCE CODE ORGANISATION

Monorepo vs. Multirepo — trade-offs & recommendations

How Should You Organize Code for Microservices?



This is one of the most debated topics in the microservices community. Three main approaches exist:

Option A



One Giant Repo, One Giant Build

All services in one repo,
one build triggers everything.
Simplest but problematic at
scale.

Option B



Multirepo (One Repo per Service)

Each microservice lives in its
own repository with its own CI
build.
Clean separation, recommended
default.

Option C



Monorepo (Multiple Builds)

All services in one repo,
but separate CI builds per
service.
Works well at small and very
large scale.

One Giant Repo, One Giant Build — The Monolithic Build



- Change 1 line in User service → rebuild everything
- Cycle time increases for all teams
- One broken service blocks ALL other services
- Hard to know which artifacts changed
- Teams fall back to deploying everything at once
- Worst of monorepo: still multiple deployables

Source Code Repo

/user-service

/catalog-service

/invoice-service

*Any commit
triggers ALL builds*

Monolithic CI Build

User build-123

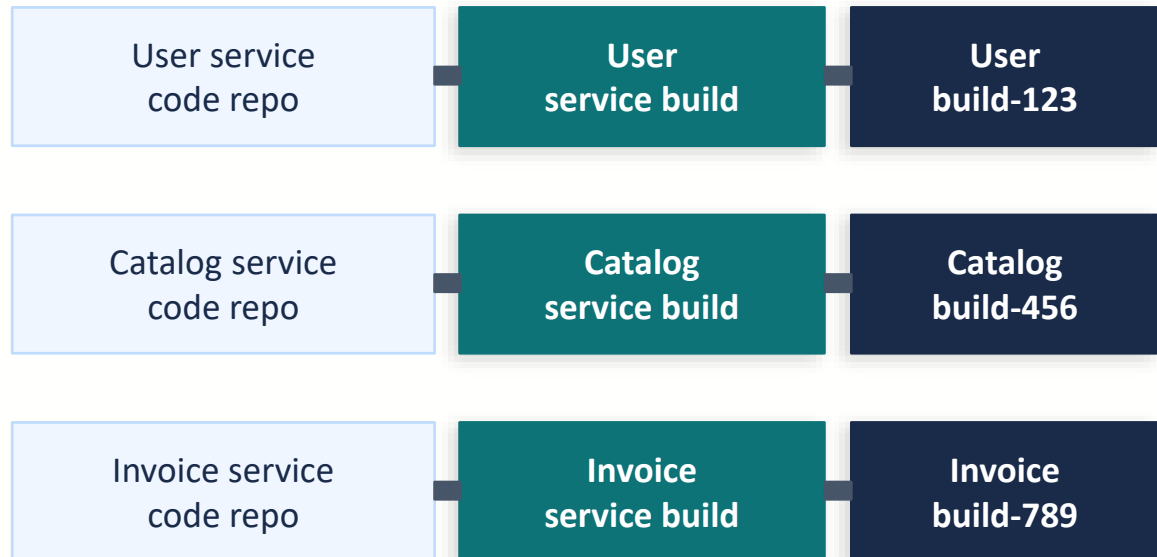
Catalog build-123

Invoice build-123

One Repository per Microservice (Multirepo)



- 1 change → 1 build → 1 artifact
- Independent deployability preserved
- Per-repo access control & ownership
- Small, focused CI configs
- Cross-service changes need 2+ commits
- Code reuse requires versioned libraries
- Juggling many repos in the IDE

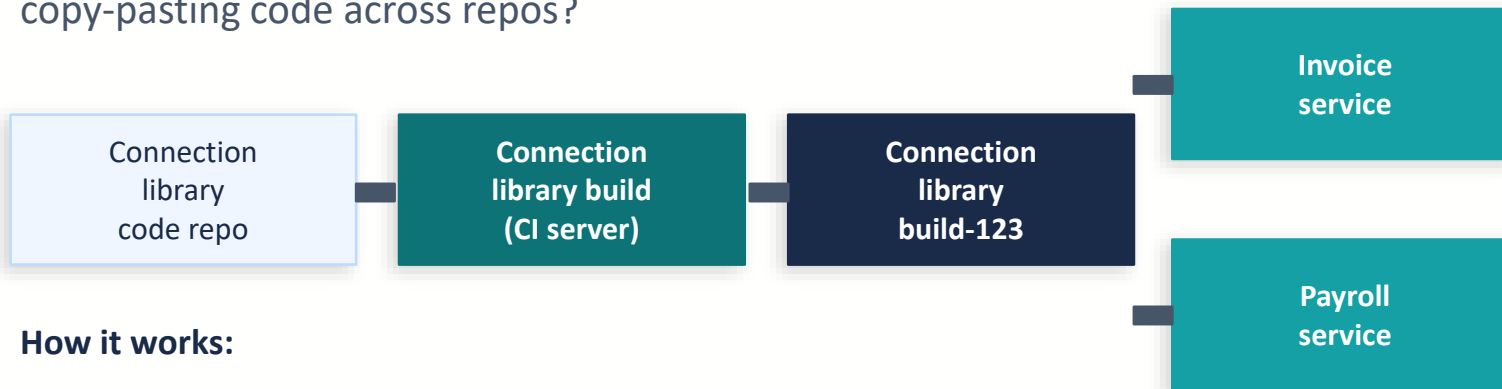


Recommended default for most teams. Clean separation, independent pipelines, easy ownership.

Reusing Code Across Repositories



When services share common logic (e.g. authentication helpers, HTTP clients), how do you avoid copy-pasting code across repos?



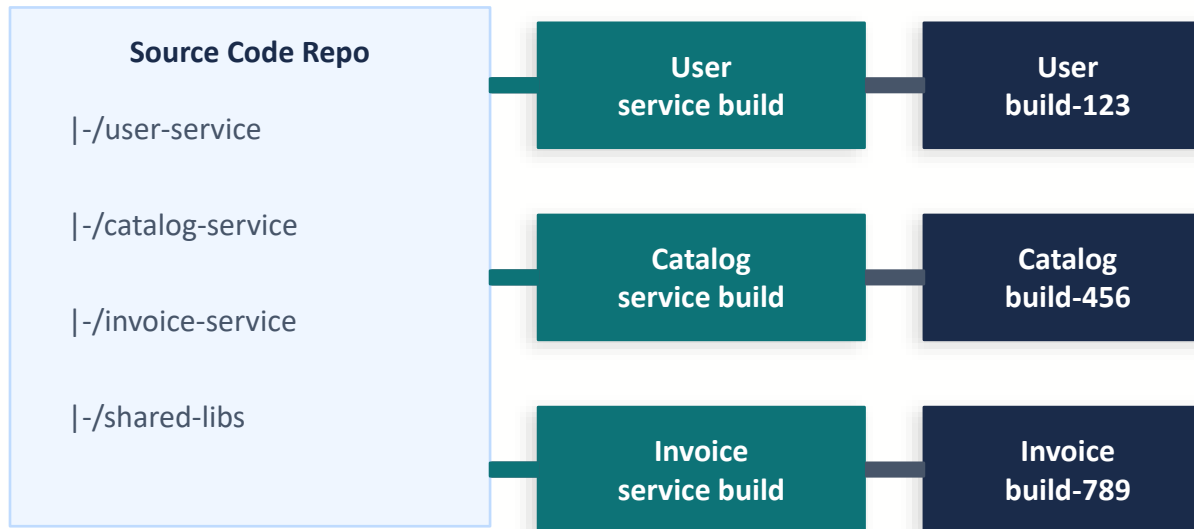
How it works:

- Package shared code as a versioned library (pip package, npm module)
- Invoice and Payroll services declare it as an explicit dependency
- A library change produces a new version artifact (e.g. connection-lib==1.2.0)
- Downstream services update their requirements.txt and redeploy
- ⚠ Deploying a new library version requires redeploying all dependent services

Pattern: Monorepo



All microservices live in one repository — but each has its own independent CI build.



Folder-based build triggers:
CI watches specific subdirectories.
Change in `/user-service` → trigger User build only.

- Pros: Atomic commits across services, fine-grained code reuse, simpler local dev
- Cons: Build triggers get complex, tooling overhead, ownership boundaries get blurry at scale
- Used by Google, Microsoft, Facebook, Uber — but they have dedicated tooling teams

Atomic Commits in a Monorepo



With a monorepo, you can change the Inventory API and the Shipping client in a single commit.



```
1 # inventory_service/api/routes.py ← server-side change
2 @app.get("/inventory/{item_id}/reserved")
3 def get_reserved_quantity(item_id: int, db: Session = Depends(get_db)):
4     reserved = db.query(Reservation).filter_by(item_id=item_id).count()
5     return {"item_id": item_id, "reserved": reserved}
6
7 # shipping_service/clients/inventory.py ← client-side change
8 async def get_reserved_quantity(item_id: int) -> int:
9     async with httpx.AsyncClient() as client:
10         resp = await client.get(f"{INVENTORY_URL}/inventory/{item_id}/reserved")
11         return resp.json()["reserved"]
```

⚠ Atomic commit ≠ Atomic rollout

Both services still need separate deployments in the right order.



Benefit: change history unified

Easier to bisect bugs that span multiple services.

Ownership in a Monorepo: CODEOWNERS



A monorepo doesn't mean everyone owns everything. GitHub (since 2016) supports a CODEOWNERS file to define owners per path.



```
1  # .github/CODEOWNERS
2
3  # Team A owns the user and catalog services
4  /user-service/      @team-platform
5  /catalog-service/   @team-catalog
6
7  # Team B owns inventory and shipping
8  /inventory-service/ @team-fulfillment
9  /shipping-service/  @team-fulfillment
10
11 # Shared libs require review from both teams
12 /shared-libs/        @team-platform @team-fulfillment
13
14 # CI configuration - DevOps team only
15 /.github/workflows/ @team-devops
```

Monorepo Build Mapping: From Simple to Complex



Mapping source folders to CI builds looks simple at first. It gets complex as shared folders appear.

✓ Simple: folder → build

```
/user-service/ → triggers User build  
/catalog-service/ → triggers Catalog build  
/invoice-service/ → triggers Invoice build
```

⚠ Complex: shared folder

```
/shared-lib/ → triggers ALL service builds  
                (because all services import from it)  
→ Needs graph-based build tool (Bazel, Nx, Turborepo)
```

- At small scale: simple folder-to-build mapping works fine
- As repos grow: shared code causes cascading rebuilds of all services
- Graph-based tools (Bazel, Nx, Turborepo, Gradle) model dependency graphs — only rebuild what actually changed
- Google's monorepo has 2B+ lines of code and required building a proprietary source control system (Piper)

Monorepo vs. Multirepo: Side-by-Side



Aspect	Monorepo	Multirepo
Atomic commits	✓ Yes, across services	✗ Requires multiple commits
Code reuse	✓ Fine-grained (single files)	⚠ Via versioned libraries only
CI build triggers	⚠ Complex (folder-based)	✓ Simple (one repo = one build)
Service isolation	⚠ Shared config risks	✓ Fully isolated pipelines
Ownership clarity	⚠ Needs CODEOWNERS files	✓ Clear per-repo boundaries
Tooling required	⚠ Bazel/Nx for large scale	✓ Standard Git + any CI tool
Developer experience	✓ One checkout for all	⚠ Multiple repo juggling
Scale sweet spot	Small (≤ 20) OR very large	Any team size
Adopters	Google, Meta, Uber (huge tooling investment)	GitHub, most mid-size companies

Which Approach to Use?



"In my experience, the main advantages of a monorepo approach — finer-grained reuse and atomic commits — don't seem to outweigh the challenges that emerge at scale. For smaller teams, either approach is fine, but as you scale, I feel that the one repository per microservice (multirepos) approach is more straightforward."

— Sam Newman

The sunk cost trap:

- Teams start small with a monorepo (collective ownership works well)
- The team grows — ownership and build complexity increase
- The monorepo becomes painful, but migrating looks expensive
- "We've invested so much in making this monorepo work..." → sunk cost fallacy

Key Takeaways

**1****CI is a practice, not just a tool**

Check in daily, have tests, fix broken builds immediately. Using Jenkins $\neq$ doing CI.

2**Trunk-Based Development beats long branches**

Integrate early and often. Use feature flags to hide WIP. Short branches if you must.

3**Build pipelines give you fast feedback and confidence**

Stage your tests: fast first, slow after. Each stage gates the next.

4**Build Once. Deploy Many.**

Create your artifact in Stage 1. Reuse it through all environments up to production.

5**Artifact must be environment-agnostic**

Config lives outside the image. Same Docker image for staging and production.

6**Multirepo is the safe default**

One service = one repo = one pipeline. Monorepo works at small or very large scale with the right tooling.

Textbooks

- Sam Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2021.



Thank You!

Questions?